L23 - MiniMind 项目简历撰写指南

> 学习目标：掌握如何将 MiniMind 项目写进简历，让面试官一眼看到你的技术深度。

前置说明

L22 已经涵盖了面试话术和 Top 20 面试题。本节聚焦于**简历文字本身**——如何用最少的字数，传递最大的信息密度，让你的简历在筛选阶段就脱颖而出。

简历和面试是两个战场：**简历决定你能不能进门，面试决定你能不能拿 Offer。** 很多人技术不差，但简历写得像流水账，连面试机会都拿不到。

一、简历项目描述的核心原则

1.1 STAR 法则在简历中的应用

STAR 法则是结构化描述项目经历的黄金框架。但在简历中，四个要素的篇幅分配需要调整——**Action 占大头，Result 紧随其后**，Situation 和 Task 用半句话带过即可。

要素	含义	简历中的体现	MiniMind 对应示例
S（Situation）	项目背景	一句话交代为什么做	大模型门槛高、个人难以全链路实践
T（Task）	你的目标	融入项目标题或第一行	从零实现完整 LLM 训练流程
A（Action）	你做了什么	每个 bullet point 的核心	实现 GQA、训练 BPE Tokenizer、完成 DPO 对齐...
R（Result）	取得了什么成果	量化数据收尾	64M 参数、单卡 3090 / ¥3 / 2h 完成训练

反面教材： Situation 写了一大段"随着人工智能技术的飞速发展....."——面试官没时间看你写作文。

正确做法： 把背景压缩到项目标题或第一行的从句中：

```
MiniMind — 从零实现的 64M 参数大语言模型全链路训练项目
```

一行就同时交代了 S（从零实现）、T（全链路训练）、R（64M 参数）。

1.2 简历描述的"三要三不要"

三要

1. 要量化成果

没有数字的简历等于没有锚点。面试官每天看几十份简历，数字是唯一能让他停下来的东西。

维度	量化示例
模型规模	64M 参数（Dense）/ 198M 参数（MoE，激活 64M）
架构细节	d_model=768, n_layers=8, q_heads=8, kv_heads=4
词表大小	BPE 词表 6400
训练成本	单卡 3090，约 ¥3 电费，约 2 小时
数据规模	预训练约 3B tokens
评测分数	C-Eval 约 24.89（64M 模型基线）
MoE 参数	4 experts / Top-1 路由，总参 198M / 激活参 64M

2. 要突出技术深度

用具体的技术名词替代泛泛描述，让面试官立刻判断你的技术层次。

模糊描述	精准描述
用了注意力机制	GQA 分组查询注意力（q=8, kv=4）
加了位置编码	RoPE 旋转位置编码（theta=1e6，支持 YaRN 外推）
做了归一化	Pre-Norm + RMSNorm
用了激活函数	SwiGLU（gate/up/down 三矩阵结构）
做了微调	LoRA（r=16，注入 Q/K/V/O 投影）

3. 要体现工程能力

面试官希望看到你不只是"跑了代码"，而是理解并能驾驭工程细节。

```
✓ 实现 loss_mask 机制，SFT 阶段仅对 assistant 回复计算交叉熵损失
✓ 使用 AMP 混合精度 + 梯度累积 + 梯度裁剪 + warmup cosine 学习率调度
✓ 基于 DDP 实现多卡数据并行训练
✗ "训练了模型"（这三个字等于什么都没说）
```

三不要

1. 不要写"参与了""了解了"这种弱动词

弱动词（扣分）	强动词（加分）
参与了模型训练	独立实现了全链路训练流程
了解了 Transformer 架构	从零构建了 Decoder-Only Transformer
学习了 DPO 算法	实现了 DPO 偏好对齐训练
使用了 LoRA 微调	基于 LoRA（r=16）完成参数高效微调，可训练参数降低至原模型 ~1%

2. 不要堆砌不相关的技术栈

如果你的 MiniMind 项目描述里出现了 Spring Boot、MySQL、Redis，面试官会怀疑你在凑字数。只写和项目直接相关的技术。

3. 不要夸大——面试会拷打每一个字

简历上写了"KV-Cache"，面试官就会问"KV-Cache 的显存占用怎么算？"。写了"DPO"，就会问"DPO 的损失函数推导一下？"。写了"MoE"，就会问"负载均衡 Loss 怎么设计？"。

铁律：简历上的每个技术名词，你都要能展开讲 2 分钟。

二、MiniMind 项目描述模板

2.1 一行版（适用于简历空间极小）

适合简历已经很满、只需要在"项目经历"中补一行的情况。

```
从零实现 64M 参数 Decoder-Only Transformer（对齐 Qwen3 架构），纯 PyTorch 完成
Tokenizer→Pretrain→SFT→LoRA→DPO/GRPO 全链路训练，单卡 3090 训练成本 ¥3/2h。
```

要点： 一行之内包含了模型规模（64M）、架构定位（Qwen3）、技术栈（PyTorch）、训练阶段（全链路）、工程能力（低成本单卡训练）。

2.2 三行版（标准版，推荐）

大多数简历的项目经历控制在 3-4 行最佳。这是信息密度和可读性的最优平衡。

```
MiniMind —— 64M 参数大语言模型全链路训练                    20XX.XX - 20XX.XX
• 基于纯 PyTorch 从零实现 Decoder-Only Transformer，对齐 Qwen3 架构
  （GQA / RoPE / RMSNorm / SwiGLU），完成 BPE Tokenizer 训练（vocab=6400）
• 实现 Pretrain(NTP, 3B tokens)→SFT(loss_mask+chat_template)→LoRA(r=16)→
  DPO→PPO/GRPO 全阶段训练，并扩展 MoE（4 experts/Top-1, 198M/A64M）及知识蒸馏
• 工程优化：AMP 混合精度 + 梯度累积 + DDP 多卡并行 + KV-Cache 推理加速；
  单卡 3090 完成全流程训练，成本 ¥3/2h
```

2.3 五行版（详细版，适用于项目经验较少时）

如果你只有 1-2 个项目，可以适当展开，用更多细节来体现深度。

```
MiniMind —— 端到端大语言模型全流程实践（纯 PyTorch 实现）        20XX.XX - 20XX.XX
• 设计并实现 Decoder-Only Transformer 架构，对齐 Qwen3：d_model=768，8 层，
  GQA（q=8, kv=4）、RoPE（theta=1e6, 支持 YaRN 长度外推）、
  Pre-Norm + RMSNorm、SwiGLU 激活，vocab=6400，embed/lm_head 权重共享，总参 64M
• 训练 BPE Tokenizer → Next Token Prediction 预训练（3B tokens）→
  SFT 多轮对话微调（loss_mask 仅计算 assistant 部分损失 + chat_template）→
  LoRA 参数高效微调（r=16, 注入 Q/K/V/O）→ DPO 偏好对齐 → PPO/GRPO 强化学习
• 扩展实现 MoE 混合专家模型（4 experts / Top-1 路由 + 负载均衡 Loss，
  总参 198M / 激活参 64M）、Tool Calling、Qwen→MiniMind 知识蒸馏（KL 散度）
• 工程优化：AMP 混合精度训练、梯度累积、梯度裁剪（max_norm=1.0）、
  warmup + cosine annealing 学习率调度、KV-Cache 推理加速、DDP 多卡数据并行
• 部署 OpenAI 兼容 API 服务，支持 ollama / vLLM / llama.cpp 推理框架，
  实现 Tool Calling 和 Adaptive Thinking；单卡 3090 全流程成本 ¥3/2h
```

2.4 技术博客 / Portfolio 版

如果你有个人博客或作品集页面，可以写更详尽的项目介绍，配合架构图和训练曲线。

```
## MiniMind —— 从零构建大语言模型

**项目定位：** 用最小的代价走完 LLM 训练全链路，每一行代码都用纯 PyTorch 实现。

**模型架构：**
Decoder-Only Transformer，64M 参数。对齐 Qwen3 架构设计：
- 隐藏维度 768，8 层 Transformer Block
- GQA 分组查询注意力：8 个 Query Head / 4 个 KV Head，head_dim=96
- RoPE 旋转位置编码（theta=1e6），支持 YaRN 长度外推至 512K tokens
- Pre-Norm + RMSNorm 归一化
```

```
- SwiGLU 激活函数（gate/up/down 三路 FFN）
- Embedding 与 LM Head 权重共享

**训练流程：**
1. BPE Tokenizer 训练（词表 6400）
2. 预训练：Next Token Prediction，约 3B tokens 中文语料
3. SFT：多轮对话微调，loss_mask + chat_template
4. LoRA：参数高效微调（r=16），仅训练 ~1% 参数
5. DPO：离线偏好对齐，无需训练 Reward Model
6. PPO/GRPO：在线强化学习对齐（Reward Model: InternLM2-1.8B-Reward）

**进阶扩展：**
- MoE 混合专家模型：4 experts / Top-1，198M 总参 / 64M 激活参
- Tool Calling：<tool_call> 标签实现工具调用
- 知识蒸馏：从 Qwen 系列大模型蒸馏到 MiniMind（黑盒 KL 散度）
- Adaptive Thinking：<think> 标签实现思维链推理

**工程实践：**
AMP 混合精度 / 梯度累积 / 梯度裁剪 / warmup + cosine LR /
KV-Cache / DDP 多卡并行 / OpenAI API 兼容部署 / ollama + vLLM + llama.cpp

**训练成本：** 单卡 RTX 3090，约 2 小时，约 ¥3 电费。
```

三、普通写法 vs 优化写法对比

以下 6 组对比覆盖了简历中最常见的问题。每一组都展示了"面试官看到后的真实反应"。

对比 1：模型架构描述

普通写法：

> 使用 PyTorch 训练了一个小型语言模型。

面试官内心： "多小？什么架构？用了什么技术？这跟调 API 有什么区别？"

优化写法：

> 基于纯 PyTorch 从零实现 64M 参数 Decoder-Only Transformer（对齐 Qwen3 架构），集成 GQA（q=8, kv=4）、RoPE、RMSNorm、SwiGLU 等现代 LLM 核心技术。

面试官内心： "这人对架构很熟，GQA 的 Q/KV 头配比都写了，可以深挖一下。"

对比 2：训练流程描述

普通写法：

> 完成了模型的预训练和微调。

面试官内心： "预训练用了多少数据？微调是全参还是 LoRA？有对齐吗？"

优化写法：

> 完成 Tokenizer(BPE, vocab=6400)→Pretrain(NTP, 3B tokens)→SFT(loss_mask+chat_template)→LoRA(r=16)→DPO→GRPO 六阶段全链路训练。

面试官内心： "全链路都走了，每个阶段的关键技术点也提了，准备问问 loss_mask 是怎么实现的。"

对比 3：工程能力描述

普通写法：

> 在 GPU 上训练了模型，进行了一些优化。

面试官内心： "什么 GPU？什么优化？这是在糊弄我吗？"

优化写法：

> 工程优化：AMP 混合精度训练 + 梯度累积（等效扩大 batch size）+ 梯度裁剪（max_norm=1.0）+ warmup cosine 学习率调度 + KV-Cache 推理加速；单卡 3090 完成全流程，成本 ¥3/2h。

面试官内心： "训练工程化做得不错，成本控制也到位，可以聊聊混合精度的细节。"

对比 4：对齐算法描述

普通写法：

> 实现了 RLHF 相关的训练。

面试官内心： "RLHF 是个很大的概念，你到底做了什么？PPO？DPO？有 Reward Model 吗？"

优化写法：

> 实现 DPO 离线偏好对齐（无需 Reward Model，直接从偏好对数据学习）和 PPO/GRPO 在线强化学习对齐（Reward Model: InternLM2-1.8B-Reward），掌握两种对齐范式的原理与工程实现差异。

面试官内心： "两种路线都做了，而且知道它们的核心区别，这个可以深入聊。"

对比 5：MoE 描述

普通写法：

> 还实现了 MoE 版本。

面试官内心： "几个专家？什么路由策略？负载均衡怎么做的？你就写了一句？"

优化写法：

> 扩展实现 MoE 混合专家架构（4 experts / Top-1 路由 + 负载均衡 Loss 防止专家坍缩），总参 198M / 激活参 64M，计算量与 Dense 版本相当。

面试官内心： "知道专家坍缩问题，知道活跃参数的概念，基础扎实。"

对比 6：部署描述

普通写法：

> 部署了模型并提供 API 服务。

面试官内心： "用了什么框架？兼容什么协议？支持什么功能？"

优化写法：

> 部署 OpenAI 兼容 API 服务，支持 ollama / vLLM / llama.cpp 多推理框架适配，实现 Tool Calling（标签）和 Adaptive Thinking（标签）功能。

面试官内心： "工程落地能力不错，多框架部署经验是加分项。"

四、不同岗位的侧重调整

同一个项目，面对不同岗位时应该突出不同的面。**不要一份简历走天下。**

4.1 大模型算法工程师

岗位核心关注： 你是否理解模型架构设计和训练对齐的原理。

突出重点： - 模型架构：Decoder-Only Transformer、GQA、RoPE、RMSNorm、SwiGLU 的设计选择和原理 - 训练全流程：Pretrain → SFT → DPO/PPO/GRPO 每个阶段的目标、损失函数、技术细节 - 对齐算法：DPO 的数学推导、PPO 的 clip 机制、GRPO 的无 Critic 设计 - Scaling Laws：参数量 vs 数据量 vs 计算量的关系

推荐关键词： `Transformer` `Decoder-Only` `GQA` `RoPE` `RMSNorm` `SwiGLU` `SFT` `DPO` `PPO` `GRPO` `RLHF` `MoE` `Scaling Laws`

三行版调整示例：

```
• 从零实现 Decoder-Only Transformer（d=768, 8层, GQA q=8/kv=4, RoPE, SwiGLU），
  对齐 Qwen3 架构，深入理解每个组件的设计动机与数学原理
• 完成 Pretrain→SFT(loss_mask)→DPO(离线偏好)→PPO/GRPO(在线 RL) 全链路对齐训练，
  掌握两种对齐范式的原理差异与工程实现
• 扩展 MoE（4 experts/Top-1 + 负载均衡 Loss），实现知识蒸馏（KL 散度），
  理解 Scaling Laws 下小模型的能力边界
```

4.2 NLP 算法工程师

岗位核心关注： 你是否有扎实的 NLP 基础和数据处理能力。

突出重点： - Tokenizer 设计：BPE 算法、词表大小选择（6400 vs 32K 的权衡）、分词粒度对模型的影响 - 数据处理：预训练数据清洗、SFT 数据格式（chat_template）、loss_mask 机制 - 评测方法：C-Eval 评测、困惑度（Perplexity）、多维度评估 - 多轮对话：对话模板设计、上下文管理

推荐关键词： `BPE` `Tokenizer` `Chat Template` `Loss Mask` `C-Eval` `Perplexity` `多轮对话` `数据清洗` `NTP` `交叉熵损失`

三行版调整示例：

```
• 基于 BPE 算法训练 6400 词表的自定义 Tokenizer，分析词表大小对 Embedding 参数占比
  和文本压缩率的影响，实现高效的中文分词方案
• 构建多阶段训练数据流水线：预训练语料清洗(3B tokens)→SFT 多轮对话数据构造
  (chat_template + loss_mask 仅计算 assistant 损失)→DPO 偏好对数据处理
• 实现多维度模型评测（C-Eval benchmark / Perplexity / 人工评估），
  分析 64M 小模型在不同任务上的能力边界与瓶颈
```

4.3 MLOps / 模型部署工程师

岗位核心关注： 你是否能把模型从训练阶段带到生产环境。

突出重点： - 训练工程化：DDP 多卡、混合精度、梯度累积、学习率调度 - 推理优化：KV-Cache、模型量化、多种解码策略（Top-K/Top-P/Temperature） - 多框架适配：ollama、vLLM、llama.cpp、GGUF 格式转换 - API 服务：OpenAI 兼容接口、流式输出

推荐关键词： `DDP` `AMP` `KV-Cache` `量化` `vLLM` `ollama` `llama.cpp` `GGUF` `OpenAI API` `流式推理` `梯度累积` `多卡训练`

三行版调整示例：

```
• 实现完整训练工程化：AMP 混合精度 + 梯度累积 + 梯度裁剪 + warmup cosine LR +
  DDP 多卡数据并行，单卡 3090 完成 64M 模型全流程训练，成本 ¥3/2h
• 推理优化：实现 KV-Cache 缓存加速（GQA 减少约 50% KV 缓存开销）、
  YaRN 长度外推(32K→512K)、多种解码策略(Top-K/Top-P/Temperature/Repetition Penalty)
• 部署 OpenAI 兼容 API 服务（流式输出），支持 ollama/vLLM/llama.cpp 多框架适配，
  实现 GGUF 模型格式转换，支持 Tool Calling 功能
```

4.4 AI 研究员 / 实习

岗位核心关注： 你是否对前沿技术有理解和实验能力。

突出重点： - 对前沿技术的理解：MoE、知识蒸馏、GRPO（DeepSeek 提出）、Agentic RL - 实验能力：消融实验、超参搜索、结果分析 - 论文关联：Chinchilla Scaling Laws、DPO 数学推导、RoPE 相对位置编码特性 - 改进方向思考：模型能力边界、训练策略优化

推荐关键词： `MoE` `知识蒸馏` `GRPO` `Scaling Laws` `消融实验` `RoPE 外推` `YaRN` `Chinchilla` `Agentic RL` `负载均衡`

三行版调整示例：

```
• 从零实现 64M 参数 LLM 全链路训练，系统性实践 Pretrain→SFT→DPO→GRPO 技术栈，
  深入理解各阶段训练目标的数学动机与工程实现差异
• 扩展实现 MoE 架构（4 experts/Top-1 + 负载均衡 Loss）和知识蒸馏
  （Qwen→MiniMind 黑盒 KL 散度），分析小模型的 Scaling Laws 特性
• 实现 RoPE + YaRN 长度外推方案，研究位置编码对长序列建模能力的影响；
  参考 Chinchilla 定律分析 64M 模型在 3B tokens 下的数据充分性
```

五、技术栈关键词清单

简历筛选阶段，HR 和技术负责人往往通过**关键词匹配**来决定是否给面试机会。以下是 MiniMind 项目可以覆盖的所有核心关键词，按类别整理。

模型架构类

关键词	写法建议
Transformer	Decoder-Only Transformer
Self-Attention	Causal Self-Attention
GQA	Grouped Query Attention（q=8, kv=4）
MHA / MQA	作为 GQA 的对比概念提及
RoPE	Rotary Position Embedding（theta=1e6）
YaRN	YaRN 长度外推（32K→512K）
RMSNorm	Pre-Norm + RMSNorm
SwiGLU	SwiGLU 激活函数（三路 FFN）
MoE	Mixture of Experts（4 experts / Top-1）
Weight Tying	embed_tokens / lm_head 权重共享

关键词	写法建议
KV-Cache	KV-Cache 推理缓存加速

训练流程类

关键词	写法建议
Pretrain	Next Token Prediction 预训练
SFT	有监督微调（loss_mask + chat_template）
LoRA	LoRA 参数高效微调（r=16）
DPO	Direct Preference Optimization
PPO	Proximal Policy Optimization
GRPO	Group Relative Policy Optimization
RLHF	Reinforcement Learning from Human Feedback
知识蒸馏	KL 散度 / 软标签蒸馏
BPE	Byte Pair Encoding Tokenizer

工程优化类

关键词	写法建议
AMP	混合精度训练（FP16 前向 + FP32 权重更新）
DDP	分布式数据并行（DistributedDataParallel）
梯度累积	等效扩大 batch size
梯度裁剪	max_norm=1.0，防止梯度爆炸
学习率调度	warmup + cosine annealing
KV-Cache	推理时缓存历史 K/V 避免重复计算
量化	模型权重量化（INT4/INT8）

部署与工具类

关键词	写法建议
vLLM	vLLM 高性能推理
ollama	ollama 本地部署
llama.cpp	llama.cpp / GGUF 格式
OpenAI API	OpenAI 兼容 API 服务
Streamlit	Web Demo 交互界面
Tool Calling	工具调用 / Function Calling
Adaptive Thinking	自适应思维链推理

评测与数据类

关键词	写法建议
C-Eval	C-Eval 中文评测基准
Perplexity	困惑度评估
多轮对话	多轮对话格式 / chat_template
偏好数据	chosen / rejected 偏好对
Reward Model	InternLM2-1.8B-Reward

> **建议：** 根据目标岗位的 JD（岗位描述）中出现的关键词，从上表中挑选对应项写入简历。不需要全写，只写你能讲清楚的。

六、简历避坑指南

6.1 绝对不要写的内容

❌ "跟着教程跑了一遍代码"

即使你确实是跟着教程学的，也不要这样写。面试官看到这句话的第一反应是"所以你的独立能力在哪？"。

正确做法：强调你在跟着学的过程中**理解了什么、思考了什么、尝试了什么变化**。

```
✗ 跟着 MiniMind 教程完成了大模型训练
✓ 基于 MiniMind 框架独立完成了全链路 LLM 训练，深入理解每个模块的设计原理
```

❌ 写了但说不清楚的技术

如果你写了"RoPE"但面试时说不出 `θ_i = 10000^{-2i/d}` 这个公式，面试官会认为你在简历造假。

原则：宁可少写一个技术点，也不要写一个你说不清楚的。 每个词都是面试时的"靶子"。

❌ 项目 Star 数

"该开源项目获得 XX K Stars" —— 这是项目作者的成就，不是你的。面试官对你参与的开源项目是否知名不感兴趣，他只关心你在其中做了什么。

❌ 复制粘贴项目 README

面试官见过无数份简历，如果你的项目描述和 README 一模一样，他会直接判定"这个人只是跑了代码"。用你自己的语言重新组织。

❌ 过度使用形容词

```
✗ 深入研究并创新性地实现了先进的注意力机制
✓ 实现 GQA 分组查询注意力（q=8，kv=4），KV-Cache 开销降低约 50%
```

6.2 面试官最关注的点

面试官在审阅你的简历时，脑子里在快速回答三个问题：

问题一：你在项目中的具体贡献是什么？

"从零实现"和"基于开源代码修改"是两码事。不需要夸大，但要明确你做了什么。

- 如果你确实是从零手写了模型代码 → 写"从零实现"
- 如果你是基于 MiniMind 代码学习并做了修改/实验 → 写"基于 MiniMind 框架实践了全链路训练，并在 [具体方面] 做了 [具体改进]"
- 如果你只是跑了代码 → 不要把这个项目写在简历上

问题二：你对技术原理的理解深度如何？

面试官会通过简历中的技术名词来判断你的技术层次：

你写的内容	面试官的判断
"使用了 Transformer"	入门级，可能只会调库
"实现了 Decoder-Only Transformer，使用 GQA 和 RoPE"	中等，知道关键技术
"实现 GQA（q=8,kv=4），RoPE（theta=1e6），Pre-Norm+RMSNorm"	较深，有实际动手经验
"对比了 GQA/MHA 在 KV-Cache 开销上的差异，实测 GQA 降低约 50% 缓存"	深入，有工程分析能力

问题三：你遇到了什么问题、怎么解决的？

这是区分"跑代码"和"做项目"的核心。如果你能在简历或面试中讲出遇到的具体问题和解决方案，说服力会大幅提升。

准备 2-3 个"踩坑-排查-解决"的小故事，例如：

- 训练 loss 不下降 → 发现学习率过大 → 添加 warmup 解决
- SFT 后模型不遵循指令格式 → 发现 loss_mask 实现有 bug → 修正后效果显著提升
- MoE 训练出现专家坍缩 → 增大负载均衡 Loss 系数 → 各专家负载趋于均匀
- PPO 训练模型崩溃 → 加大 KL 惩罚 + 降低学习率 → 稳定训练

6.3 简历上每个词都要能展开讲 2 分钟

这是黄金法则。在提交简历之前，做以下测试：

逐词检验法： 指着简历上的每一个技术名词，问自己：

> "如果面试官问我 [这个技术] 是什么，我能不能流畅地讲 2 分钟？"

如果不能，要么把这个词删掉，要么去学明白。

模拟追问法： 对于每个技术点，准备好"纵向深挖三层"的回答：

```
第一层：是什么？
  "RoPE 是旋转位置编码，通过旋转矩阵将位置信息编码到 Q 和 K 中。"

第二层：怎么实现的？
  "把向量维度两两配对，每对用一个频率 θ_i = base^{-2i/d}。
   位置 m 的旋转角度是 m×θ_i，用复数乘法高效实现。"
```

```
第三层：为什么这样设计？
  "相对位置信息体现在 Q 和 K 内积中（旋转角度变成差 (m-n)×θ_i），
   不需要显式计算相对位置。并且天然支持外推到更长序列。"
```

七、简历自查清单

提交简历前，逐条检查以下 10 项。每一项都打上 ✅ 才算合格。

✅ Checklist

#	检查项	说明
1	**有具体数字吗？**	参数量（64M）、词表（6400）、训练成本（¥3/2h）、数据量（3B tokens）等至少出现 3 个量化指标
2	**技术名词精准吗？**	不是"注意力机制"而是"GQA（q=8, kv=4）"；不是"位置编码"而是"RoPE"
3	**动词够强吗？**	使用"实现""构建""设计""优化"，避免"参与""了解""学习"
4	**每个技术名词都能讲 2 分钟吗？**	指着简历逐词检验，不能讲的就删掉
5	**突出了全链路吗？**	Tokenizer→Pretrain→SFT→LoRA→DPO/GRPO 完整链路是核心竞争力，不要漏掉
6	**有工程能力体现吗？**	AMP、梯度累积、DDP、KV-Cache 等工程优化至少提到 2-3 项
7	**和目标岗位匹配吗？**	根据 JD 中的关键词调整侧重点（参考第四节）
8	**没有夸大或造假吗？**	确保写的每一点都经得起面试追问
9	**格式整洁、信息密度高吗？**	每行都有信息量，没有废话和填充词，标点和缩进统一
10	**时间线合理吗？**	项目时间段标注合理，不与其他项目/实习时间冲突

自查流程

```
第 1 遍：内容检查（对照上表 10 项）
第 2 遍：语言检查（删除废话、替换弱动词、增加量化数据）
```

```
第 3 遍：模拟面试（找朋友/同学指着简历随机追问）
第 4 遍：岗位适配（针对每个目标岗位微调侧重点）
```

八、完整简历示例

以下是一个完整的简历项目经历示例，带有逐行批注说明每行的设计意图。

示例：应聘大模型算法工程师

```
项目经历
──────────────────────────────────────────────────────────────

MiniMind —— 64M 参数大语言模型全链路训练            20XX.03 - 20XX.06
技术栈：Python / PyTorch / Transformers / DDP / vLLM

• 基于纯 PyTorch 从零实现 Decoder-Only Transformer（对齐 Qwen3），          ← ①
  配置 d_model=768、8 层、GQA（q=8, kv=4）、RoPE（theta=1e6）、
  Pre-Norm + RMSNorm、SwiGLU 激活，vocab=6400，总参数 64M

• 完成六阶段全链路训练：BPE Tokenizer 训练 → Next Token Prediction          ← ②
  预训练（3B tokens）→ SFT 多轮对话微调（loss_mask 仅计算 assistant
  部分损失）→ LoRA 参数高效微调（r=16）→ DPO 离线偏好对齐 →
  PPO/GRPO 在线强化学习（Reward Model: InternLM2-1.8B-Reward）

• 扩展实现 MoE 混合专家模型（4 experts / Top-1 路由 + 负载均衡 Loss，       ← ③
  198M 总参 / 64M 激活参）、Qwen→MiniMind 知识蒸馏（黑盒 KL 散度）、
  Tool Calling 及 Adaptive Thinking 功能

• 工程优化：AMP 混合精度 + 梯度累积 + cosine LR 调度 + KV-Cache 推理       ← ④
  加速 + DDP 多卡并行；部署 OpenAI 兼容 API，适配 ollama/vLLM/llama.cpp；
  单卡 3090 完成全流程训练，成本约 ¥3/2h
```

逐行批注

标记	设计意图
标题行	项目名 + 一句话定位（64M、全链路），时间段标注清晰
技术栈行	快速告诉面试官核心技术栈，方便关键词匹配
① 架构	用"从零实现""对齐 Qwen3"建立技术可信度；列出所有架构关键技术并带上具体参数（d=768, GQA q=8/kv=4, theta=1e6），展示你对细节的掌握

标记	设计意图
② 训练	用箭头连接六个阶段，一眼看出完整链路；每个阶段括号内标注关键技术点（loss_mask、r=16、InternLM2-1.8B-Reward），信息密度极高
③ 扩展	展示超越基础训练的进阶能力：MoE（带具体配置）、蒸馏（带具体方法）、Tool Calling，说明你不是只做了"基本操作"
④ 工程	一行覆盖训练优化 + 部署 + 成本，最后用"¥3/2h"这个数字收尾，给面试官留下深刻印象

示例：应聘 NLP 算法工程师（侧重点调整）

```
MiniMind —— 端到端大语言模型训练与评测实践                20XX.03 - 20XX.06
技术栈：Python / PyTorch / BPE / SFT / DPO

• 基于 BPE 算法训练 6400 词表的自定义 Tokenizer，分析词表大小对 Embedding
  参数占比（6400×768≈5M，占总参数约 8%）和文本压缩率的影响

• 构建多阶段数据流水线：3B tokens 中文预训练语料清洗 → SFT 多轮对话数据
  构造（chat_template 格式 + loss_mask 仅计算 assistant 损失）→ DPO 偏好
  对数据处理（chosen/rejected 格式）

• 实现 64M 参数 Decoder-Only Transformer（GQA/RoPE/RMSNorm/SwiGLU），
  完成 Pretrain→SFT→LoRA→DPO→GRPO 全链路训练

• 搭建多维度评测体系：C-Eval 中文基准测试（得分 24.89）、Perplexity 评估、
  多轮对话质量人工评测，分析 64M 小模型的能力边界
```

示例：应聘 MLOps / 部署工程师（侧重点调整）

```
MiniMind —— 大语言模型训练工程化与多框架部署              20XX.03 - 20XX.06
技术栈：Python / PyTorch / DDP / vLLM / ollama / llama.cpp / OpenAI API

• 实现完整训练工程化：AMP 混合精度（FP16 前向 + FP32 权重更新）+ 梯度累积
  （等效扩大 batch size）+ 梯度裁剪（max_norm=1.0）+ warmup + cosine LR
  调度 + DDP 多卡数据并行

• 推理优化：实现 KV-Cache 缓存加速（GQA 配置减少约 50% KV 缓存显存）、
  YaRN 长度外推（32K→512K tokens）、Top-K/Top-P/Temperature 等多种解码策略

• 部署 OpenAI 兼容 RESTful API 服务（支持流式输出），实现 GGUF 模型格式
  转换，适配 ollama / vLLM / llama.cpp 三种推理框架

• 实现 Tool Calling（<tool_call> 标签解析）和 Adaptive Thinking（<think>
  标签控制思维链输出），Streamlit Web Demo 交互界面
```

附录：简历常见 Q&A

Q：我只跑了部分流程（比如只做了 Pretrain 和 SFT），能写全链路吗？

A：不能。只写你实际做了的部分。写"完成了 Pretrain → SFT 两阶段训练"完全没问题，不需要硬凑。面试官问到 DPO 时你答不上来，比不写 DPO 更糟糕。

Q：我是跟着教程学的，能写"从零实现"吗？

A：关键不在于代码是否从零手敲，而在于你是否**理解了每一行代码**。如果你能脱离教程独立解释每个模块的设计动机和实现细节，就可以写"从零实现"。如果只是 copy-paste 跑通了，就不要写。

Q：MiniMind 是开源项目，面试官会不会觉得"大家都做了"？

A：同一个项目，不同人写出来的简历差距巨大。面试官看的是你的理解深度和表达方式，不是项目本身。能讲清楚"为什么 GQA 用 8Q/4KV 而不是 8Q/2KV""为什么词表选 6400 而不是 32K"这类设计权衡问题的人，永远不会和"跑了一遍代码"的人混淆。

Q：如果我在 MiniMind 基础上做了改进（改了超参、加了新功能），怎么写？

A：这是最佳情况。把你的改进作为一个单独的 bullet point 突出：

```
• 在 MiniMind 基础上进行 [具体改进]：[做了什么] → [达到了什么效果]
  例如：将 GQA 配置从 q=8/kv=4 调整为 q=8/kv=2 进行消融实验，
  KV-Cache 显存降低 25%，C-Eval 分数变化在 1% 以内
```

Q：简历上的技术栈行应该写什么？

A：只写和项目直接相关的核心技术，按"语言 / 框架 / 核心技术 / 部署工具"的顺序排列：

```
✓ Python / PyTorch / Transformers / DDP / vLLM
✗ Python / PyTorch / TensorFlow / Keras / Scikit-learn / Pandas / NumPy / ...
  （不要把不相关的技术栈堆上去）
```

> **最后的话：** 简历是你的"推理输出"——要像训练 LLM 一样对待它：数据要高质量（内容真实、有深度）、格式要标准（结构清晰、关键词覆盖）、loss 要低（每一行都有信息量，没有废话）。把你对 MiniMind 的理解浓缩到一页纸上，让面试官在 10 秒内做出"必须见一见"的判断。